

## 표적 궤적 모의

표적을 정의하고, 0.1초마다 움직이고, 위경도로 그리기까지

개념 · 좌표변환 코드 이용 · 표적 구조체 · 궤적 모의 구현 · 결과

레이다 시스템 소프트웨어 스터디 · Part 3

### 발표 대본

표지 · 약 20초

안녕하십니까. 과제 3, 표적 궤적 모의 발표를 시작하겠습니다. 지난 과제에서는 레이다가 잦 값을 위경도로 바꾸는 일을 했는데, 이번에는 반대로 표적을 우리가 직접 정의해서 60초 동안 움직여 보는 일입니다.

과제가 제출하라고 한 항목이 네 가지입니다. 좌표변환 코드를 어떻게 썼는지, 표적 구조체를 어떻게 정의했는지, 궤적을 어떻게 모의했는지 코드로, 그리고 결과가 어떻게 나왔는지. 이 네 가지를 순서대로 한 장씩 다루고, 그 앞에 필요한 개념만 짧게 짚겠습니다.

## INDEX

## 목차

|    |            |                               |
|----|------------|-------------------------------|
| 00 | 과제 요구사항    | 네 가지 제출 항목과 이 자료의 대응          |
| 01 | 필요한 개념     | 좌표계 · 위치와 속도<br>시간 적분 · 기동 모델 |
| 02 | 좌표변환 코드 이용 | 함수 넷 · 인자 순서와 단위<br>프로그램 구성   |
| 03 | 표적 구조체 정의  | 구조체 셋 · 항목 ↔ 칸<br>값 채우기       |
| 04 | 궤적 모의 구현   | 다섯 단계 · 함수별 코드<br>결과 화면       |
| 05 | 시뮬레이션 결과   | 실행 화면 · 궤적과 고도<br>기동 · 검증     |

02 ~ 05 가 과제 제출 항목 (1) ~ (4) 에 그대로 대응합니다. 개념은 코드를 읽는 데 필요한 만큼만.

## 발표 대본

목차 · 약 40초

여섯 부분입니다.

들어가며에서 과제 요구사항을 이 자료의 어느 장에서 답하는지 대응시켜 놓고 시작합니다.

첫째 장은 필요한 개념입니다. 좌표계 넷, 위치와 속도를 왜 다르게 다루는지, 시간을 어떻게 흘려보내는지, 기동을 몇  $g$  라는 숫자 하나로 어떻게 표현하는지. 네 장입니다.

둘째 장부터가 과제 제출 항목입니다. 좌표변환 코드에서 어떤 함수를 어디에 썼고, 인자 순서와 단위를 어떻게 맞췄고, 프로그램이 어떻게 구성되는지.

셋째 장은 표적 구조체입니다. 코드를 그대로 보여 드리고, 과제가 요구한 항목이 어느 칸에 들어갔는지 표로 대응시킵니다.

넷째 장은 구현입니다. 한 스텝을 다섯 단계로 나누고, 초기화, 속도 변환, 한 스텝 함수, 시나리오 루프, 화면까지 코드로 따라갑니다.

다섯째 장은 결과입니다. 실행 화면, 궤적과 고도 그림, 기동 시나리오, 그리고 결과가 맞는지 확인한 검증표입니다.

## 00. 과제 3 요구사항과 이 자료의 대응

표적을 정의하고, 0.1초마다 움직이고, 위경도로 그려라. 제출 항목 넷은 각각 한 장(章)으로



## 구조체 · 갱신 · 출력 조건

표적 최대 10 개, 표적마다 기동 최대 30 개

기동 = Gravity Value · TurnType (0 없음 · 1 Roll · 2 Yaw · 3 Pitch) · StartTime · EndTime

위치·속도는 ECEF 에서 갱신, 결과는 LLA 로 출력하고 그림으로

플랫폼은 정지 (속력 0). 이 프로그램은 플랫폼도 같은 표적 구조체로 다룬다.

## 시뮬레이션 시간

60 초 · 0.1 초 간격

- $t = 0.0 \text{ s}$
- $t = 0.1 \text{ s}$
- $t = 0.2 \text{ s}$
- $t = 0.3 \text{ s}$
- $\vdots$
- $t = 60.0 \text{ s}$

## 위치 갱신 600

표적마다 · 위경도로 기록

| 과제가 제출하라고 한 것        | 이 자료           |
|----------------------|----------------|
| (1) 좌표변환 코드 이용       | 02 장 · 10 ~ 12 |
| (2) 표적 구조체 정의 설명     | 03 장 · 14 ~ 16 |
| (3) 궤적 모의 구현 방법을 코드로 | 04 장 · 18 ~ 23 |
| (4) 시뮬레이션 결과 설명      | 05 장 · 25 ~ 28 |

### 시뮬레이션 조건

플랫폼 32.0°N 126.0°E. 정지

대함 표적 32.125°N 126.03°E, 고도 0 m, 30 m/s, yaw 270° (서쪽)

대공 표적 32.12°N 126.0°E, 고도 300 m, 200 m/s, yaw 180° (남쪽)

시간 60 s, 간격 0.1 s

자세각은 roll · pitch · yaw 전부 0° 에서 시작. 기동 목록은 과제에 없어 예시를 정해 넣었다 (8장).

## 발표 대본

들어가며 · 약 70초

과제를 한 줄로 줄이면, 표적을 정의하고, 0.1초마다 움직이고, 위경도로 그려라입니다.

왼쪽 그림이 상황입니다. 우리 배는 북위 32도, 동경 126도에 서 있고 움직이지 않습니다. 북쪽으로 13.3 킬로미터 떨어진 곳에 대공 표적이 고도 300 미터에서 초속 200 미터로 남쪽, 그러니까 우리 쪽으로 날아옵니다. 그 옆에 대함 표적이 초속 30 미터로 서쪽으로 갑니다. 60초 동안 0.1초 간격이니 표적마다 600개의 점이 찍힙니다.

오른쪽 표가 과제가 제출하라고 한 네 가지와 이 자료의 대응입니다. 좌표변환 코드 이용은 둘째 장, 표적 구조체 정의 설명은 셋째 장, 궤적 모의 구현 방법을 코드로 설명하는 것은 넷째 장, 시뮬레이션 결과 설명은 다섯째 장입니다. 질문하실 때 이 번호로 찾아오시면 됩니다.

그 아래가 시뮬레이션 조건입니다. 표적은 최대 10개, 표적마다 기동은 최대 30개, 상태는 ECEF 좌표계에서 갱신, 60초를 0.1초 간격으로, 결과는 위경도로 출력하고 그림으로. 이 조건은 전부 코드에 숫자 그대로 들어가 있습니다.

## 01

필요한  
개념

- 1 좌표계 넷, 그리고 왜 ECEF 인가
- 2 위치와 속도는 다르게 변환한다 — 회전 두 번
- 3 시간 적분과 지구 곡률 — 60초 직진하면 11 m 떠오른다
- 4 기동 모델 — 하중배수 · 세 축 · 시간표

약속 : 각도는 도(°) 로 말하고 프로그램 안에서는 라디안. 거리는 m, 시간은 s.

4

## 발표 대본

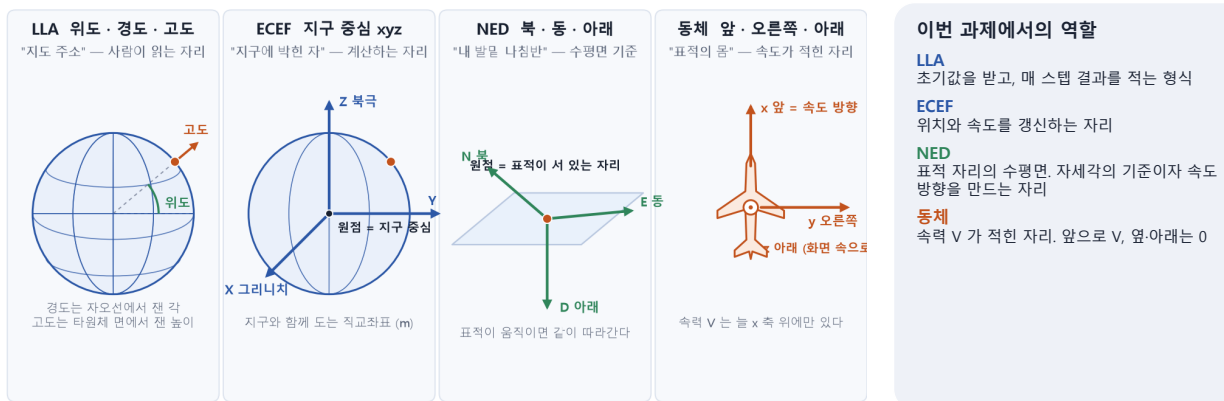
장절 · 약 25초

첫째 장, 개념입니다. 코드를 보기 전에 네 가지만 정리하고 가겠습니다. 어느 좌표계를 쓰는지, 위치와 속도가 왜 다른지, 시간을 어떻게 더하는지, 그리고 기동을 어떻게 숫자로 나타내는지입니다.

약속 하나만 드리면, 발표에서는 각도를 도로 말하고 프로그램 안에서는 라디안을 씁니다. 거리는 전부 미터, 시간은 초입니다.

## 01. 좌표계 넷, 그리고 왜 ECEF 인가

입력: 출력은 위경도, 계산은 ECEF, 수평면은 NED, 속력은 동체



왜 ECEF 인가 : 위도  $1^\circ = 111 \text{ km}$ , 경도  $1^\circ = 94 \text{ km}$  ( $32^\circ\text{N}$ ) — 각도에 미터를 더할 수 없다. ECEF 는 세 축이 미터라  $p + v\Delta t$  한 줄 플랫폼과 무관한 절대 좌표라, 나중에 배가 움직여도 표적 궤적이 흔들리지 않는다.

5

## 발표 대본

개념 · 약 80초

지난번에 여섯 개를 봤는데 이번에 쓰는 건 넷입니다.

첫째, 위도 경도 고도. 사람이 읽는 자리입니다. 과제도 처음 위치를 이 형식으로 받고 결과도 이 형식으로 내라고 합니다.

둘째, ECEF. 지구 중심에 원점을 두고 미터로 x, y, z 를 재는 직교좌표입니다. 계산은 전부 여기서 합니다.

셋째, NED. 표적이 서 있는 자리의 북쪽, 동쪽, 아래쪽입니다. 원점이 플랫폼이 아니라 표적 자신이라는 게 중요합니다. 자세각이 이 수평면 기준으로 정의되니까요.

넷째, 동체. 표적의 몸 기준으로 앞, 오른쪽, 아래입니다. 과제가 표적 속도를 앞으로 가는 속력 하나로 받으라고 했는데, 그게 적혀 있는 자리가 여기입니다.

왜 ECEF 에서 움직이느냐. 위도는 각도입니다. 북위  $32^\circ$  근처에서 위도  $1^\circ$  는  $111 \text{ km}$  인데 경도  $1^\circ$  는  $94 \text{ km}$  입니다. 같은  $1^\circ$  라는 눈금인데 길이가 다르고, 애초에 도에다 미터를 더할 수가 없습니다. ECEF 는 세 축이 전부 미터라 지금 위치에 속도 곱하기 시간을 더하면 끝입니다. 그리고 플랫폼과 무관한 절대 좌표라, 나중에 배가 움직이는 시나리오가 와도 표적 궤적은 그대로입니다.

## 01. 위치와 속도는 다르게 변환한다 — 회전 두 번

위치는 원점이 있어야 말이 되고, 속도는 화살표 하나면 된다. 그래서 속도는 회전만



속도는 화살표라서 두 번 다 회전만 한다. ③ 에서 위도·경도는 플랫폼이 아니라 표적이 지금 있는 자리의 값이다.

**위치:**  $p\_new = R \cdot p + t$

원점이 다른 숫자가 다르다.  
회전하고 원점 차이만큼 옮긴다 (과제 2).

**속도:**  $v\_new = R \cdot v$

어디에 그러도 같은 화살표.  
축이 돌아간 만큼만 돌린다. t 를 더하면 속도가 아니게 된다.

**검산**

결과 벡터의 크기는 항상 V (200.000).  
플랫폼 위치를 잘못 더하면 6,372 km/s.

**과제의 "추가 고민" (동체 속도  $V\_heading \rightarrow ECEF$ ) 의 답:** 자세각으로 한 번, 표적 자신의 위도·경도로 한 번, 평행이동은 없다  
f\_Trans\_Ned\_To\_Ecef 는 플랫폼 위치를 더하게 되어 있으므로, 속도를 넣을 때는 그 인자에 0 을 준다.

6

## 발표 대본

개념 · 약 80초

이번 과제의 핵심 하나입니다. 과제에 추가 고민 항목으로 나온, 동체 속도  $V\_heading$  을 ECEF 성분으로 바꾸는 방법의 답이기도 합니다.

위치는 원점이 있어야 성립합니다. 같은 점을 다른 원점에서 보면 숫자가 다르니, 좌표계를 바꿀 때 회전하고 원점 차이만큼 옮깁니다. 지난 과제에서 한 일입니다.

속도는 화살표입니다. 어디에 그러도 같은 화살표라 원점이 필요 없습니다. 축이 돌아간 만큼만 돌리면 되고, 여기에 원점 차이를 더하면 속도가 아니게 됩니다.

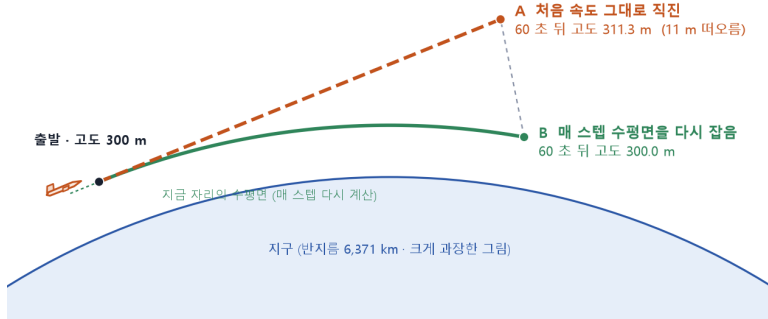
그래서 그림처럼 두 번 다 회전만 합니다. 동체의 앞 방향 속력  $V$  를 자세각으로 한 번 돌려 NED 성분으로 만들고, 표적 자신의 위도·경도로 한 번 더 돌려 ECEF 성분으로 만듭니다. 대공 표적이면 앞으로 200 이 남쪽으로 200 이 되고, 다시 ECEF 세 성분이 되는데 크기는 여전히 200 입니다.

검산이 간단합니다. 결과 벡터의 크기가 항상 입력 속력이어야 합니다. 참고 코드의 NED 에서 ECEF 함수는 플랫폼 위치를 더하게 되어 있어서, 속도를 넣을 때는 그 인자에 0 을 줘야 합니다. 실수로 위치를 넣으면 초속 6,372 킬로미터가 나옵니다. 지구 반지름이 속도에 섞인 겁니다.

## 01. 시간 적분과 지구 곡률

새 위치 = 지금 위치 + 속도 × 0.1 초. 단, 매 스텝 수평면을 다시 잡지 않으면 60초에 11 m 떠오른다

왜 떠오르나 : 지구가 둥글어서 접선 방향으로 12 km 가면 땅이 11 m 아래로 내려간다. ( $d^2 / 2R$ )  
그래서 매 스텝 표적의 현재 위경도로 NED 수평면을 다시 잡고, 그 위에서 속도 방향을 다시 만든다



시간 적분 :  $p(t+\Delta t) = p(t) + v(t) \cdot \Delta t$

ECEF x, y, z 에 각각.  $\Delta t = 0.1$  s.

한 스텝 안에서 속도가 일정하니 이 한 줄이면 정확.

실험 (대공 표적 60초 직진)

60초 뒤 고도

A 처음 ECEF 속도를 60초 내내 그대로

311.3 m

B 매 스텝 현재 위경도로 NED 를 다시 잡음

300.0 m

왜 11 m 인가

접선을 따라 d 만큼 가면 땅은  $d^2 / 2R$  만큼 내려간다.  
 $12,000^2 / (2 \times 6,371,000) = 11.3$  m. 실험값과 일치.

그래서 5단계 LLA 변환은 출력용만이 아니라 다음 스텝의 수평면(NED 축) 을 만드는 기준이다

위경도를 구해야 NED 축이 나오고, NED 축이 있어야 속도 방향이 나온다. 이 순서를 지키면 수평 비행이 수평으로 유지된다.

7

## 발표 대본

개념 · 약 80초

시간을 흘러보내는 방법은 한 줄입니다. 새 위치는 지금 위치에 속도 곱하기 0.1초를 더한 것. ECEF x, y, z 에 각각 더하면 되고, 한 스텝 안에서 속도가 일정하니 이 한 줄이면 정확합니다. 이것 60초 동안 반복합니다.

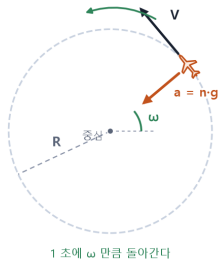
그런데 함정이 하나 있습니다. 지구가 둥글다는 겁니다. 오른쪽 표를 보시면, 대공 표적을 60초 직진시켰을 때 처음 구한 ECEF 속도를 60초 내내 그대로 쓰면 고도가 300 미터에서 311 미터로 떠오릅니다. 접선을 따라 12 킬로미터를 가면 땅이 그만큼 내려가니까요. d 제곱 나누기 2R, 12,000 제곱 나누기 2 곱하기 6,371,000 이 11.3 미터입니다. 실험값과 같습니다.

해결은 매 스텝 표적이 지금 있는 자리의 수평면을 다시 잡는 겁니다. 그러면 60초 뒤 고도가 300.0 미터로 유지됩니다.

그래서 한 스텝의 마지막 단계인 위경도 변환이 출력용만이 아닙니다. 위경도를 구해야 다음 스텝의 NED 축이 나오고, NED 축이 있어야 속도 방향이 나옵니다. 이 순서를 지키면 수평 비행이 수평으로 유지됩니다.

## 01. 기동 모델 — 하중배수, 세 축, 시간표

과제의 Gravity Value 는 "몇 g 로 도는가". 속력은 그대로, 방향만 바뀐다. TurnType 은 동체의 어느 축을 돌리는가



## 동속 원운동 공식

구심가속도  
 $a = V^2 / R = n \cdot g$

선회 반경  
 $R = V^2 / (n \cdot g)$

각속도  
 $\omega = V / R = n \cdot g / V$

$g = 9.80665 \text{ m/s}^2$   
 $V$  는 변하지 않는다

$$\omega = n \cdot g / V, \quad R = V^2 / (n \cdot g)$$

$a = V^2 / R = n \cdot g$ . 프로그램에서는 매 스텝 자세각  $\pm \omega \times 0.1 \text{ s}$  (3 g 면 한 스텝에 0.84°)

## 감 잡기

1 g 서 있을 때, 0.3 g 승용차 커브, 3 g 전투기 급선회, 9 g 한계. 각속도가 속력에 반비례 — 배 기동에는 0.1 g 같은 작은 값.

| 표적         | 하중배수  | 각속도       | 선회 반경   | 비고          |
|------------|-------|-----------|---------|-------------|
| 대공 200 m/s | 1 g   | 2.81 °/s  | 4,079 m | 180° 에 64 s |
| 대공 200 m/s | 3 g   | 8.43 °/s  | 1,360 m | 180° 에 21 s |
| 대공 200 m/s | 5 g   | 14.05 °/s | 816 m   | 180° 에 13 s |
| 대함 30 m/s  | 0.1 g | 1.87 °/s  | 918 m   | 90° 에 48 s  |

| TurnType | 동체 축    | 효과              | 과제 예시                              |
|----------|---------|-----------------|------------------------------------|
| 0        | —       | 직진              | 기동 없음                              |
| 1 Roll   | x (앞뒤)  | 자세만 기울 · 방향 그대로 | —                                  |
| 2 Yaw    | z (위아래) | 좌우 선회           | 대공 +3 g 10~20 s, 대함 -0.1 g 20~50 s |
| 3 Pitch  | y (날개)  | 상승 · 강하         | 대공 +2 g 35~40 s                    |

## 시간표 규칙 셋

StartTime ≤ t < EndTime 이면 적용, 밖이면 직진. 부호 : n < 0 이면 반대 방향. 겹침 : 먼저 적힌 것 하나. 끝난 뒤 : 자세 유지 (안 건드리는 것이 곧 유지).

8

## 발표 대본

개념 · 약 90초

기동입니다. 과제의 Gravity Value 는 몇 g 로 도는가입니다. 원운동에서 안쪽으로 당기는 가속도가  $n g$  이면 반지름은  $V$  제곱 나누기  $n g$ , 각속도는  $n g$  나누기  $V$  입니다. 가속도가 속도에 수직이라 속력은 안 변하고 방향만 변합니다.

오른쪽 위 표가 감을 잡기 위한 숫자입니다. 초속 200 미터 대공 표적이 3 g 로 돌면 초당 8.4도, 반지름 1,360 미터입니다. 초속 30 미터 배가 0.1 g 로 돌면 초당 1.9도, 반지름 918 미터. 각속도가 속력에 반비례하니 배 기동에는 작은 g 를 넣어야 그럴듯합니다.

TurnType 은 동체의 어느 축을 돌리는가입니다. 1이 롤, 앞뒤 축. 2가 요, 위아래 축이라 좌우 선회. 3이 피치, 날개 축이라 상승 강하. 0은 직진입니다.

시간표 규칙은 셋입니다. 시작 시각 이상 끝 시각 미만이면 적용하고 밖이면 직진. g 의 부호가 방향이라 음수면 반대로. 기동이 겹치면 먼저 적힌 것 하나만. 그리고 기동이 끝난 뒤에는 그 자세를 유지합니다.

프로그램에서는 한 줄입니다. 각속도  $n g$  나누기  $V$  를 구해 놓고, 매 스텝 자세각에 각속도 곱하기 0.1초를 더합니다. 3 g 면 한 스텝에 0.84도입니다. 축을 돌린 뒤 속도가 어느 쪽인지는 따로 계산하지 않습니다. 자세각만 바뀌 두면 앞 장의 회전 두 번이 새 속도 방향을 만들어 줍니다.

과제 조건에는 기동 목록이 없어서, 발표용 예시 기동을 정해 넣었습니다. 대공 3 g 우선회 10초에서 20초, 2 g 기수 들기 35초에서 40초, 대함 마이너스 0.1 g 좌선회 20초에서 50초입니다.

## 02

좌표변환  
코드 이용

- 1 참고 소스 코드와 이번에 쓰는 함수 넷
- 2 인자 순서와 단위 — 컴파일해서 확인한 규약
- 3 프로그램 구성

좌표변환은 참고 소스 코드의 함수를 그대로 부르고, 인자 순서와 단위는 부르는 쪽에서 맞췄다.

9

## 발표 대본

장절 · 약 25초

둘째 장, 과제 제출 항목의 첫째인 좌표변환 코드 이용입니다.

세 장으로 보겠습니다. 참고 소스 코드 다섯 파일 중 이번에 부르는 함수 넷이 무엇이고 어느 단계에서 쓰는지, 컴파일해서 확인한 인자 순서와 단위 규약, 그리고 프로그램이 어떻게 구성되는지입니다.

## 02. 참고 소스 코드와 이번에 쓰는 함수 넷

좌표변환 14 함수 중 넷을 부른다. 초기화에 하나, 매 스텝 속도 변환에 둘, 매 스텝 출력에 하나

## 참고 소스 코드 (5 파일)

CoordinateTransform.c / .h  
좌표변환 14 함수 (WGS-84, 각도 라디안)  
matrixCalcLib.c / .h  
3×3 행렬 곱·합 (변환 함수 안에서 쓰임)  
Define.h  
DOUBLE64 · INT32 · VOID, PI, G\_FORCE 9.80665

| 함수 (인자)                                                   | 하는 일                        | 어디서                      |
|-----------------------------------------------------------|-----------------------------|--------------------------|
| f_Trans_Lla_To_Ecef (Lon, Lat, Alt)                       | 위경도·고도 → ECEF 위치            | 초기화 (f_InitTarget) 한 번   |
| f_Trans_Body_To_Ned (x, y, z, Roll, Yaw, Pitch)           | 동체 벡터 → NED 벡터 (자세각 회전)     | ③ 속도 변환, 매 스텝            |
| f_Trans_Ned_To_Ecef (n, e, d, Pf_X, Pf_Y, Pf_Z, Lat, Lon) | NED 벡터 → ECEF 벡터 (+ 플랫폼 위치) | ③ 속도 변환, 매 스텝 (Pf = 0)   |
| f_Trans_Ecef_To_Lla (X, Y, Z)                             | ECEF 위치 → 위경도·고도            | ⑤ 매 스텝 끝, 출력 + 다음 NED 기준 |

한 스텝에서 넷이 이어지는 순서 (초기화 한 번은 위경도 → f\_Trans\_Lla\_To\_Ecef → ECEF 위치)



나머지 열 함수 (안테나 계열 4 · ENU 2 · 플랫폼 보정 · ECEF→NED · Body↔Ant) 는 이번 과제에서 부르지 않는다  
표적을 만드는 쪽이라 안테나 좌표계가 없고, 플랫폼이 서 있어서 플랫폼 보정도 없다.

10

## 발표 대본

좌표변환 코드 · 약 90초

참고 소스 코드는 다섯 파일입니다. CoordinateTransform 이 좌표변환 함수 14개, matrixCalcLib 이 3 곱하기 3 행렬 곱셈과 덧셈, Define.h 가 타입 이름과 상수입니다. 원주율과 중력가속도 9.80665 도 여기 있어서 그대로 씁니다.

14개 함수 중 이번에 부르는 건 넷입니다. 표를 보시면, 위경도 고도를 ECEF 위치로 바꾸는 함수는 초기화에서 한 번. 동체 벡터를 자세각으로 돌려 NED 벡터로 만드는 함수와 NED 벡터를 ECEF 벡터로 만드는 함수는 매 스텝 속도 변환에서. ECEF 위치를 위경도로 되돌리는 함수는 매 스텝 마지막에. 이렇게 넷입니다.

아래 흐름이 그 넷이 한 스텝에서 이어지는 모습입니다. 동체의  $V$  가 NED 속도가 되고 ECEF 속도가 되고, 위치에 더해지고, 위경도로 돌아옵니다. 그 위경도가 다음 스텝의 NED 기준이 됩니다. 앞 장에서 본 곡률 처리가 이 고리입니다.

나머지 열 개, 안테나 좌표계 넷, ENU 둘, 플랫폼 보정, ECEF 에서 NED 로 가는 함수는 이번엔 부르지 않습니다. 표적을 만드는 쪽이라 안테나 좌표계가 없고, 플랫폼이 서 있어서 보정도 없습니다.

## 02. 인자 순서와 단위 — 컴파일해서 확인한 규약

헤더만 읽고 넘어가지 않고 하나씩 값을 찍어 봤다. 틀리면 어떤 값이 나오는지도 같이

| 함수                  | 인자 순서 · 단위                                           | 틀리면                                    |
|---------------------|------------------------------------------------------|----------------------------------------|
| f_Trans_Lla_To_Ecef | 경도, 위도, 고도 — 경도가 첫째 인자. 라디안 · m                      | 위도-경도를 바꿔 넣으면 북위 54° 동경 212°           |
| f_Trans_Body_To_Ned | (x, y, z) 다음 Roll, Yaw, Pitch — Roll-Pitch-Yaw 가 아니다 | yaw 와 pitch 가 바뀌어 대공 표적이 땅으로 쏜다        |
| f_Trans_Ned_To_Ecef | (n, e, d), 플랫폼 ECEF 위치 셋, 플랫폼 위도·경도 — 속도엔 위치에 0      | 플랫폼 위치를 넣으면 $ v  = 6,372 \text{ km/s}$ |
| f_Trans_Ecef_To_Lla | X, Y, Z [m] → Lat, Lon [rad], Alt [m]. 5회 고정 반복      | 반복 부족이면 고도 오차 — 우리 위치에서 왕복 3e-5 m 로 충분 |
| 공통                  | 각도는 전부 라디안. DEG2RAD / RAD2DEG 매크로는 target_sim.h 에 정의 | 도(°) 를 그대로 넣으면 32 rad = 1,833°         |

## target\_sim.c 가 좌표변환 함수를 부르는 네 줄 (pstTgt-&gt; 생략)

```
// 초기화 (f_InitTarget) : 경도가 첫째 인자
stPos = f_Trans_Lla_To_Ecef(stInitLla.Lon, stInitLla.Lat, stInitLla.Alt);

// ③ 동체 → NED (f_BodyVelToEcef) : 자세각 순서 Roll, Yaw, Pitch
stNed = f_Trans_Body_To_Ned(dvheading, 0.0, 0.0, stAtt.Roll, stAtt.Yaw, stAtt.Pitch);

// ④ NED → ECEF : 속도라서 플랫폼 위치 인자(가운데 셋) 에 0. 위경도는 표적 자신의 것
stVel = f_Trans_Ned_To_Ecef(stNed.x, stNed.y, stNed.z, 0.0, 0.0, 0.0, stLla.Lat, stLla.Lon);

// ⑤ ECEF → LLA (f_StepTarget) : 출력이자 다음 스텝의 NED 기준
stLla = f_Trans_Ecef_To_Lla(stPos.x, stPos.y, stPos.z);
```

## 컴파일해서 확인한 것

- ① Body→NED 행렬 =  $R_z(\text{yaw}) \cdot R_y(\text{pitch}) \cdot R_x(\text{roll})$  과 소수점 6자리 일치 (과제 2 규약)
- ② LLA → ECEF → LLA 왕복 오차 3e-5 m
- ③ 속도 변환 결과  $|v| = 200.000000 = V$
- ④ Pf 를 넣으면 6,372 km/s → 속도엔 0 이 맞다

11

## 발표 대본

좌표변환 코드 · 약 90초

좌표변환 함수를 쓸 때 조심한 것들입니다. 헤더만 읽고 넘어가지 않고 하나씩 컴파일해서 값을 찍어 봤습니다.

첫째, 위경도를 ECEF 로 바꾸는 함수는 경도가 첫째 인자입니다. 위도 경도 순서로 넣으면 북위 54도 동경 212도라는 이상한 자리가 나옵니다. 둘째, 동체에서 NED 로 가는 함수의 자세각 순서는 롤, 요, 피치입니다. 롤 피치 요가 아닙니다. 셋째, NED 에서 ECEF 로 가는 함수는 가운데 인자 셋이 플랫폼 위치인데, 속도를 넣을 때는 여기에 0 을 줍니다. 넷째, 각도는 전부 라디안입니다. 도 단위 변환 매크로는 제 헤더에 정의했습니다.

아래 왼쪽이 코어에서 실제로 부르는 네 줄입니다. 인자 순서가 표와 같은지 한 줄씩 대조하실 수 있습니다.

오른쪽이 확인한 결과입니다. 동체에서 NED 로 가는 행렬을 직접 곱한 Rz 요, Ry 피치, Rx 롤 과 비교해서 소수점 여섯 자리까지 같았습니다. 지난 과제와 같은 규약입니다. 위경도를 ECEF 로 갔다가 되돌리는 왕복 오차는 0.03 밀리미터, 참고 코드가 반복을 다섯 번 고정으로 하는데 이 정도면 충분합니다. 속도 변환 결과의 크기는 정확히 200 이었고, 플랫폼 위치를 넣어 보니 초속 6,372 킬로미터가 나와서 0 을 주는 게 맞다는 걸 확인했습니다.

## 02. 프로그램 구성

좌표변환은 참고 소스 코드의 함수가 하고, C 코어가 표적을 움직이고, MFC 화면은 결과를 보여 준다

## 참고 소스 코드

C · 좌표변환

CoordinateTransform.c / .h  
좌표변환 14 함수 (WGS-84)  
matrixCalcLib.c / .h  
행렬 곱 · 합  
Define.h  
타입 · 상수 (PI, G, FORCE)

링크

## TargetSimCore (C 라이브러리)

이번에 짰 부분 · target\_sim.h / .c

ST\_Manever · ST\_Target · ST\_Scenario  
구조체 셋  
f\_SetAssignment  
과제 조건 채우기  
f\_StartScenario · f\_StepScenario  
초기화, 0.1 초 한 스텝

호출

## TargetSimUI (MFC 대화상자)

들리고 보여 주기만 · C++

[실행]  
60 초를 돌려 위경도를 모은다  
지도 · 슬라이더 · 제생  
궤적과 임의 시각의 위치  
표 · [CSV 저장]  
위경도 출력

좌표변환은 전부 참고 소스 코드의 함수가 하고, 코어는 그 함수를 순서대로 부르지만 한다. MFC 는 계산을 하지 않는다.

| 코어의 함수 다섯       | 하는 일                                | 부르는 곳            |
|-----------------|-------------------------------------|------------------|
| f_SetTarget     | 표적 카드 한 장의 입력 칸을 채운다 (각도는 도로 받는다)   | f_SetAssignment  |
| f_AddManeuver   | 기동 하나를 목록 끝에 붙인다                    | f_SetAssignment  |
| f_SetAssignment | 과제 조건 : 플랫폼 · 대함 · 대공, 60 s · 0.1 s | 화면 [실행]          |
| f_StartScenario | 입력 칸으로 상태 칸을 채운다 (위경도 → ECEF)       | 화면 [실행], 한 번     |
| f_StepScenario  | 플랫폼과 표적 전부를 0.1 초 진행한다              | 화면 [실행], 0.1 초마다 |

12

## 발표 대본

좌표변환 코드 · 약 70초

프로그램 구성입니다. 세 덩어리입니다.

왼쪽이 참고 소스 코드 다섯 파일입니다. 좌표변환은 전부 이 코드의 함수가 합니다. 가운데가 이번에 짰 C 코어, target\_sim.h 와 .c 입니다. 구조체 셋과 함수 다섯이 있고, 좌표변환 함수를 순서대로 부르지만 합니다. 이 둘을 묶어 C 라이브러리로 만들었습니다. 오른쪽이 MFC 대화상자입니다. 계산이 없습니다. 코어를 돌려 위경도를 모으고, 지도와 표로 보여 주고, CSV 로 저장합니다.

아래 표가 코어의 함수 다섯입니다. 표적 카드에 값을 넣는 f\_SetTarget, 기동을 붙이는 f\_AddManeuver, 과제 조건을 채우는 f\_SetAssignment, 시작하는 f\_StartScenario, 한 스텝 가는 f\_StepScenario. 화면이든 콘솔이든 이 다섯만 부르면 됩니다. 뒤 장에서 하나씩 보겠습니다.

## 03

표적 구조체  
정의

- 1 구조체 셋 — `target_sim.h`
- 2 과제 항목이 들어간 칸, 그리고 설계 결정
- 3 값 채우기 — `f_SetTarget` · `f_AddManeuver` · `f_SetAssignment`

입력 칸은 사람이 적고, 상태 칸은 프로그램이 갱신한다. 플랫폼도 카드 한 장.

13

## 발표 대본

장절 · 약 20초

셋째 장, 표적 구조체입니다. 기동 하나, 표적 하나, 시나리오 하나, 이렇게 구조체 셋을 코드 그대로 보여 드리고, 과제가 요구한 항목이 어느 칸에 들어갔는지 표로 대응시킨 뒤, 값을 채우는 함수를 보겠습니다.

## 03. 구조체 셋 — target\_sim.h

위 다섯 칸이 입력(사람이 적는 값), 아래 네 칸이 상태(프로그램이 갱신). 좌표 타입은 참고 소스 코드의 구조체 그대로

표적 카드 #2

ST\_Target

## 입력 (사람이 적는 칸)

초기 위치

32.12°N 126.0°E 300 m

동체 속도

200 m/s

초기 자세

roll 0° pitch 0° yaw 180°

기동 목록 (최대 30 개)

| g | Type  | Start | End |
|---|-------|-------|-----|
| 3 | Yaw   | 10    | 20  |
| 2 | Pitch | 35    | 40  |
| - |       |       |     |

## 상태 (프로그램이 갱신)

ECEF 위치

x, y, z [m]

ECEF 속도

vx, vy, vz [m/s]

현재 자세

roll, pitch, yaw

현재 위경도

출력용 · 매 스텝 변환

0.1 s

매 스텝 갱신 · 600 번

입력 칸은 건드리지 않는다

## 플랫폼도 속도 0 인 표적 카드

갱신 함수가 하나로 통일되고, 배가 움직이는 시나리오가 와도 카드 값만 바뀌면 된다. MAX\_TARGET 10, MAX\_MANEUVER 30.

```
typedef struct
{
    DOUBLE64 dGravity; // 하중배수 [g]. 부호가 방향
    INT32 nTurnType; // 0 없음 1 Roll 2 Yaw 3 Pitch
    DOUBLE64 dStartTime; // [s]
    DOUBLE64 dEndTime; // [s]
} ST_Maneuver;

typedef struct
{
    /* 입력 */
    STRUCT_Coord_Lla stInitLla; // 초기 위치 [rad, rad, m]
    DOUBLE64 dVheading; // 동체 속도 [m/s]
    STRUCT_Coord_Attitude stInitAtt; // 초기 자세 [rad]
    INT32 nManeuverCnt; // 기동 수
    ST_Maneuver astManeuver[MAX_MANEUVER]; // 30
    /* 상태 (ECEF 에서 갱신) */
    STRUCT_Coord_Rect stPos; // ECEF 위치 [m]
    STRUCT_Coord_Rect stVel; // ECEF 속도 [m/s]
    STRUCT_Coord_Attitude stAtt; // 현재 자세 [rad]
    STRUCT_Coord_Lla stLla; // 현재 위경도 (출력용)
} ST_Target;

typedef struct
{
    ST_Target stPlatform; // 속도 0 인 표적 카드
    INT32 nTargetCnt;
    ST_Target astTarget[MAX_TARGET]; // 10
    DOUBLE64 dSimTime; // 60 s
    DOUBLE64 dDt; // 0.1 s
} ST_Scenario;
```

14

## 발표 대본

구조체 · 약 90초

왼쪽 그림처럼 표적 하나를 카드 한 장으로 생각하면 편합니다. 오른쪽이 헤더의 실제 코드입니다.

기동 하나가 ST\_Maneuver 입니다. 하중배수, 축, 시작, 끝. 과제가 준 네 항목 그대로입니다.

표적 하나가 ST\_Target 이고 두 칸으로 나뉩니다. 위쪽 다섯 줄이 입력 칸입니다. 초기 위치를 위경도와 고도로, 속력을 동체 앞 방향 하나로, 초기 자세를 롤 피치 요로, 그리고 기동 목록을 배열 30칸과 개수로. 과제가 요구한 입력 형식이 그대로 칸이 됩니다.

아래 네 줄이 상태 칸입니다. ECEF 위치, ECEF 속도, 현재 자세, 그리고 출력용 현재 위경도. 과제가 ECEF 에서 갱신하라고 한 그 값들입니다. 처음에 입력 칸을 읽어 상태 칸을 채우고, 그 뒤로는 상태 칸만 고칩니다.

시나리오가 ST\_Scenario 입니다. 플랫폼 카드 한 장, 표적 카드 최대 10장과 개수, 시뮬레이션 시간과 간격. 플랫폼도 속도 0인 표적 카드로 둔 게 요령입니다. 갱신 함수가 하나로 통일되고, 배가 움직이는 시나리오가 와도 카드 값만 바뀌면 됩니다.

좌표 타입은 참고 소스 코드의 구조체를 그대로 썼습니다. 위경도, 직교좌표, 자세각 셋 다 이미 있으니까요.

## 03. 과제 항목이 들어간 칸, 그리고 설계 결정

과제가 요구한 항목 하나에 칸 하나. 좌표 타입은 참고 소스 코드의 것, 단위는 라디안 · m · s

| 과제 요구 항목                  | 구조체 칸                               | 타입 · 단위                      |
|---------------------------|-------------------------------------|------------------------------|
| 초기 위경도 · 고도               | ST_Target.stInitLla                 | STRUCT_Coord_Lla [rad,rad,m] |
| 동체 속력 V_heading           | ST_Target.dVheading                 | DOUBLE64 [m/s]               |
| 자세각 roll · pitch · yaw    | ST_Target.stInitAtt                 | STRUCT_Coord_Attitude [rad]  |
| 기동 Gravity Value          | ST_Maneuver.dGravity                | DOUBLE64 [g], 부호 = 방향        |
| 기동 TurnType 0 / 1 / 2 / 3 | ST_Maneuver.nTurnType               | INT32 TURN_xxx 매크로           |
| 기동 StartTime · EndTime    | ST_Maneuver.dStartTime · dEndTime   | DOUBLE64 [s]                 |
| 표적 최대 10 개                | ST_Scenario.astTarget[MAX_TARGET]   | #define MAX_TARGET 10        |
| 표적마다 기동 최대 30 개           | ST_Target.astManeuver[MAX_MANEUVER] | #define MAX_MANEUVER 30      |
| ECEF 갱신 위치 · 속도           | ST_Target.stPos · stVel             | STRUCT_Coord_Rect [m],[m/s]  |

동적 할당 없음 — 배열 크기는 과제 조건 10 · 30 을 #define 으로  
표적을 늘리려면 숫자 하나. 초기화 뒤에는 상태 칸만 바뀌므로 같은 입력이면 언제 돌려도 같은 결과.

## ① 입력 칸과 상태 칸을 나눴다

초기화 때 입력을 한 번 읽어 상태를 만들고, 그 뒤로는 상태만. 다시 실행해도 입력이 그대로다.

## ② 좌표 타입은 참고 소스 코드의 구조체 그대로

STRUCT\_Coord\_Lla · Rect · Attitude 를 칸으로 쓰니  
좌표변환 함수에 바로 넣고 바로 받는다.

## ③ 프로그램 안은 라디안 · m · s

도(°) 는 값을 넣는 f\_SetTarget 에서만 받아 DEG2RAD.  
화면에 보일 때만 RAD2DEG.

## ④ 플랫폼도 카드 한 장 (속력 0)

ST\_Scenario.stPlatform. 같은 초기화 · 같은 스텝 함수를 지난다.  
배가 움직이는 조건이 오면 속력과 yaw 만 넣으면 된다.

15

## 발표 대본

구조체 · 약 90초

과제 요구 항목을 구조체 칸에 하나씩 대응시킨 표입니다.

초기 위경도와 고도는 stInitLla, 참고 코드의 위경도 구조체이고 단위는 라디안과 미터입니다. 동체 속력 V\_heading 은 dVheading, 초당 미터. 자세각은 stInitAtt, 참고 코드의 자세각 구조체. 기동의 Gravity Value 는 dGravity, TurnType 은 nTurnType 으로 0 없음 1 롤 2 요 3 피치, 시작 끝 시각은 dStartTime 과 dEndTime. 표적 최대 10개는 시나리오의 배열 크기, 기동 최대 30개는 표적 안의 배열 크기입니다. 그리고 ECEF 에서 갱신하라는 위치와 속도가 stPos, stVel 로 참고 코드의 직교좌표 구조체입니다.

오른쪽이 설계 결정 셋입니다. 첫째, 입력과 상태를 나눴습니다. 초기화 때 한 번 입력을 읽어 상태를 만들고, 그 뒤엔 상태만 갱신합니다. 다시 실행해도 입력이 그대로라 같은 결과가 나옵니다. 둘째, 좌표 타입은 참고 코드의 구조체를 그대로 칸으로 썼습니다. 변환 함수에 바로 넣고 바로 받습니다. 셋째, 프로그램 안은 라디안, 미터, 초로 통일하고, 도 단위는 값을 넣는 함수 하나에서만 받습니다.

배열은 고정 크기입니다. 과제 조건 10 과 30 을 define 으로 두었고 동적 할당은 없습니다. 표적을 더 넣으려면 숫자 하나만 바꾸면 됩니다.

## 03. 값 채우기 — f\_SetTarget · f\_AddManeuver · f\_SetAssignment

각도는 도로 받아 함수 안에서 라디안으로. 체크박스를 켜면 발표용 기동 예시가 붙는다

```
VOID f_SetAssignment(ST_Scenario *pstScn, INT32 bWithManeuver)
{
    memset(pstScn, 0, sizeof(*pstScn));
    pstScn->dSimTime = 60.0;
    pstScn->dDt = 0.1;
    pstScn->nTargetCnt = 2;

    /* 위도 경도 고도 속력 roll yaw pitch */
    f_SetTarget(&pstScn->stPlatform, 32.0, 126.0, 0.0, 0.0, 0.0, 0.0, 0.0);
    f_SetTarget(&pstScn->astTarget[0], 32.125, 126.03, 0.0, 30.0, 0.0, 270.0, 0.0); // 대함
    f_SetTarget(&pstScn->astTarget[1], 32.12, 126.0, 300.0, 200.0, 0.0, 180.0, 0.0); // 대공

    if (bWithManeuver) // 발표용 기동 예시
    {
        f_AddManeuver(&pstScn->astTarget[0], -0.1, TURN_YAW, 20.0, 50.0); // 대함 좌선회
        f_AddManeuver(&pstScn->astTarget[1], 3.0, TURN_YAW, 10.0, 20.0); // 대공 3 g 우선회
        f_AddManeuver(&pstScn->astTarget[1], 2.0, TURN_PITCH, 35.0, 40.0); // 대공 기수 돌기
    }

    // f_SetTarget : 각도는 도로 받아 라디안으로 저장 (프로그램 안은 라디안 · m · s)
    pstTgt->stInitLat = DEG2RAD(dLatDeg); ... pstTgt->stInitAtt.Yaw = DEG2RAD(dYawDeg);
    // f_AddManeuver : 목록 끝에 붙인다. nManeuverCnt >= MAX_MANEUVER 면 무시
}
```

|         | 플랫폼          | 표적 1 대함         | 표적 2 대공       |
|---------|--------------|-----------------|---------------|
| 위도 · 경도 | 32.0 · 126.0 | 32.125 · 126.03 | 32.12 · 126.0 |
| 고도 m    | 0            | 0               | 300           |
| 속력 m/s  | 0            | 30              | 200           |
| yaw     | 0            | 270 (서)         | 180 (남)       |

## 기동 예시 (화면의 체크박스)

대함 : -0.1 g yaw 20~50 s (음수 = 좌선회)  
 대공 : +3 g yaw 10~20 s, +2 g pitch 35~40 s  
 표적을 더 넣으려면 f\_SetTarget 줄을 더 쓴다 (최대 10).

시간 60 s · 간격 0.1 s · 표적 2 개 — 과제 시뮬레이션 조건 그대로

16

## 발표 대본

구조체 · 약 70초

구조체에 값을 넣는 함수입니다.

f\_SetTarget 이 표적 카드 한 장의 입력 칸을 채웁니다. 위도, 경도, 고도, 속력, 롤, 요, 피치를 받는데 각도는 도로 받아서 함수 안에서 라디안으로 바꿔 저장합니다. f\_AddManeuver 가 기동 하나를 목록 끝에 붙입니다. 30개를 넘으면 무시합니다.

f\_SetAssignment 가 과제 조건입니다. 시간 60초, 간격 0.1초, 표적 둘. 플랫폼은 북위 32도 동경 126도, 속력 0. 대함 표적은 북위 32.125도 동경 126.03도, 고도 0, 초속 30 미터, 요 270도라 서쪽. 대공 표적은 북위 32.12도 동경 126.0도, 고도 300 미터, 초속 200 미터, 요 180도라 남쪽. 오른쪽 표가 같은 값입니다.

화면의 체크박스를 켜면 발표용 기동 예시가 붙습니다. 대함 마이너스 0.1 g 요 20초에서 50초, 음수라 좌선회. 대공 플러스 3 g 요 10초에서 20초, 플러스 2 g 피치 35초에서 40초. 표적을 더 넣으려면 f\_SetTarget 줄을 더 쓰면 됩니다.

## 04

궤적 모의  
구현

- 1 한 스텝의 다섯 단계
- 2 초기화와 속도 변환 — `f_InitTarget` · `f_BodyVelToEcef`
- 3 한 스텝 함수 ①② — 기동 판단 · 자세 갱신
- 4 한 스텝 함수 ③④⑤ — 속도 · 위치 · 위경도
- 5 시나리오 전체 — `f_StartScenario` · `f_StepScenario`
- 6 결과 모으기와 화면 — `TargetSimUI` (MFC)

함수 하나에 한 장. 코드는 `target_sim.c` 그대로다.

17

## 발표 대본

장절 · 약 25초

넷째 장, 궤적 모의 구현입니다. 코드로 설명하라는 항목이라 여섯 장에 걸쳐 함수를 하나씩 따라가겠습니다.

먼저 한 스텝을 다섯 단계로 나눈 그림을 보고, 초기화와 속도 변환 함수, 한 스텝 함수를 두 장에 나눠, 시나리오 전체를 돌리는 함수, 그리고 결과를 모아 보여 주는 화면 순서입니다.

## 04. 한 스텝의 다섯 단계

표적 하나가 0.1초 뒤로 가는 일. 함수 하나가 이 순서 그대로이고, 0.1 초마다 표적 수만큼 반복한다

## 표적 하나의 한 스텝 (0.1 초)

왼쪽부터 순서대로. ① ② 는 기동이 없으면 건너뛰고, ③ ④ ⑤ 는 매 스텝 반드시 한다.



① 에 필요한 것  
기동 목록, 지금 시각 t

② 에 필요한 것  
각속도  $\omega = n-g / V, \Delta t$

③ 에 필요한 것  
자세각 3개, 표적 위경도

④ 에 필요한 것  
ECEF 위치·속도,  $\Delta t = 0.1 s$

⑤ 에 필요한 것  
ECEF → LLA 변환 함수

순서가 중요한 건 ③ ④ ⑤ — 자세가 정해져야 속도, 속도가 있어야 위치, 위치가 있어야 새 수평면. 플랫폼과 표적 모두 같은 순서

18

## 발표 대본

구현 · 약 60초

표적 하나가 0.1초 뒤로 가는 일을 다섯 단계로 나눴습니다. 함수 하나가 이 순서 그대로입니다.

첫째, 기동 판단. 지금 시각이 어느 기동 구간인지 봅니다. 둘째, 자세 갱신. 구간 안이면 각속도 곱하기 0.1 초만큼 그 축의 자세각을 돌립니다. 이 둘은 기동이 없으면 건너뛵니다.

셋째, 속도 변환. 동체 속력을 NED 로, ECEF 로 두 번 돌립니다. 넷째, 위치 적분. ECEF 위치에 속도 곱하기 0.1 초를 더합니다. 다섯째, 위경도 변환. 기록하고, 다음 스텝의 수평면 기준으로 씁니다. 이 셋은 매 스텝 반드시 합니다.

순서가 중요한 건 셋째 넷째 다섯째입니다. 자세가 정해져야 속도, 속도가 있어야 위치, 위치가 있어야 새 수평면입니다.

## 04. 초기화와 속도 변환 — f\_InitTarget · f\_BodyVelToEcef

6장의 회전 두 번이 열 줄. 좌표변환 함수에서 조심할 점 두 가지가 이 화면에 다 있다

## ① 동체 → NED

f\_Trans\_Body\_To\_Ned 에 [V, 0, 0] 과 자세각.  
인자 순서는 Roll, Yaw, Pitch (헤더에 적힌 대로).

## ② NED → ECEF

f\_Trans\_Ned\_To\_Ecef 의 가운데 셋이 플랫폼 위치.  
속도는 벡터라 여기에 0. 위경도는 표적 자신의 것.

## 초기화의 함정

f\_Trans\_Lla\_To\_Ecef 는 (경도, 위도, 고도) 순서.  
바꿔 넣으면 북위 54° 동경 212° 가 나온다.

검산 :  $|v_{ecef}| = 200.000000 \text{ m/s}$

실수로 플랫폼 위치를 더하면 6,372 km/s

```
// 동체 속력 [V, 0, 0] 을 ECEF 속도로. 회전 두 번, 평행이동 없음
static STRUCT_Coord_Rect f_BodyVelToEcef(const ST_Target *pstTgt)
{
    STRUCT_Coord_Rect stNed;

    // 1) 동체 → NED : 자세각으로 회전 (인자 순서 Roll, Yaw, Pitch)
    stNed = f_Trans_Body_To_Ned(pstTgt->dVheading, 0.0, 0.0,
                                pstTgt->stAtt.Roll, pstTgt->stAtt.Yaw, pstTgt->stAtt.Pitch);

    // 2) NED → ECEF : 표적 자신의 위경도로 회전.
    // 속도는 벡터라 플랫폼 위치 인자(가운데 셋) 에 0
    return f_Trans_Ned_To_Ecef(stNed.x, stNed.y, stNed.z, 0.0, 0.0, 0.0,
                                pstTgt->stLla.Lat, pstTgt->stLla.Lon);
}

// 초기화 : 입력 칸 → 상태 칸. f_Trans_Lla_To_Ecef 는 인자 순서가 (경도, 위도, 고도)
static VOID f_InitTarget(ST_Target *pstTgt)
{
    pstTgt->stPos = f_Trans_Lla_To_Ecef(pstTgt->stInitLla.Lon,
                                        pstTgt->stInitLla.Lat, pstTgt->stInitLla.Alt);
    pstTgt->stAtt = pstTgt->stInitAtt;
    pstTgt->stLla = pstTgt->stInitLla;
    pstTgt->stVel = f_BodyVelToEcef(pstTgt);
}
```

19

## 발표 대본

구현 · 약 80초

오른쪽 아래가 초기화 함수입니다. 입력 칸의 위경도를 좌표변환 함수로 ECEF 위치로 바꾸고, 자세와 위경도를 상태 칸에 복사하고, 속도를 한 번 계산해 둡니다. 여기서 인자 순서가 경도, 위도, 고도라는 걸 다시 보실 수 있습니다.

위가 속도 변환 함수입니다. 앞에서 본 회전 두 번이 열 줄입니다.

첫째 줄, 동체에서 NED 로. 변환 함수에 V, 0, 0 과 자세각 롤, 요, 피치를 넣습니다. 헤더에 적힌 순서 그대로입니다.

둘째 줄, NED 에서 ECEF 로. 이 함수의 가운데 인자 셋이 플랫폼 위치인데 속도는 벡터라 0 을 줍니다. 마지막 두 인자, 위도 경도는 플랫폼이 아니라 표적 자신의 현재 위경도입니다. 이게 매 스텝 달라지기 때문에 이 함수를 매 스텝 다시 부릅니다.

검산은 결과 벡터의 크기입니다. 200.000000, 입력 속력 그대로입니다.

## 04. 한 스텝 함수 f\_StepTarget — ①② 기동 판단과 자세 갱신

기동 목록을 훑어 지금 시각의 구간을 찾고, 그 축을  $\omega \times 0.1$  s 만큼 돌린다. 구간 밖이면 아무것도 안 한다

```
static VOID f_StepTarget(ST_Target *pstTgt, DOUBLE64 dTime, DOUBLE64 dDt)
{
    INT32 i;

    // 1) 기동 판단, 2) 자세 갱신 : 각속도  $\omega = n \cdot g / V$ 
    for (i = 0; i < pstTgt->nManeuverCnt; i++)
    {
        const ST_Maneuver *pstMan = &pstTgt->astManeuver[i];

        if ((dTime + 1e-6 >= pstMan->dStartTime) && (dTime + 1e-6 < pstMan->dEndTime)
            && (pstTgt->dVheading > 0.0))
        {
            DOUBLE64 dAng = pstMan->dGravity * G_FORCE / pstTgt->dVheading * dDt;

            if (pstMan->nTurnType == TURN_ROLL)    pstTgt->stAtt.Roll += dAng;
            else if (pstMan->nTurnType == TURN_YAW) pstTgt->stAtt.Yaw += dAng;
            else if (pstMan->nTurnType == TURN_PITCH) pstTgt->stAtt.Pitch += dAng;
            break;
            // 먼저 적용된 기동 하나만
        }
    }
    // ... @ @ @ 는 다음 장
}
```

기동 하나 = { Gravity Value, TurnType, StartTime, EndTime } 표적마다 최대 30 개



구간 판정 :  $Start \leq t < End$

+1e-6 은 0.1 을 거듭 더할 때 생기는 부동소수점 오차 여유.  
시작 10.0 인 기동이 한 스텝 늦지 않게.

각속도 한 줄 :  $dAng = n \cdot G\_FORCE / V \cdot dt$

G\_FORCE 9.80665 는 Define.h 의 상수.  
3 g, 200 m/s, 0.1 s → 0.0147 rad = 0.84°.

축 선택과 break

TurnType 대로 Roll · Yaw · Pitch 하나에 더한다.  
break — 겹치면 먼저 적용된 기동 하나만.

속력 0 이면 건너뛴

플랫폼도 같은 함수를 지나는데  $\omega = n \cdot g / V$  의 분모가 V.  
속력 0 인 카드는 기동 판단을 건너뛰고 자리만 지킨다.  
끝난 뒤 자세 유지 = 안 건드리는 것.

20

## 발표 대본

구현 · 약 80초

한 스텝 함수의 앞부분, 기동 판단과 자세 갱신입니다.

for 문으로 기동 목록을 처음부터 훑습니다. 조건이 셋입니다. 지금 시각이 시작 시각 이상, 끝 시각 미만, 그리고 속력이 0 보다 클 것. 속력 0 조건은 플랫폼 때문입니다. 플랫폼도 같은 함수를 지나는데 각속도 식에 속력이 분모라서요.

시각 비교에 1e-6 을 더한 건 부동소수점 때문입니다. 0.1 을 거듭 더하면 정확히 10.0 이 아니라 9.9999999 가 될 수 있어서, 시작 시각 10 인 기동이 한 스텝 늦게 시작하지 않도록 아주 작은 여유를 줬습니다.

조건에 맞으면 각속도  $n \cdot g$  나누기  $V$  에  $dt$  를 곱한 각도를 구하고, TurnType 에 따라 롤, 요, 피치 중 하나에 더합니다. G\_FORCE 는 Define.h 의 9.80665 입니다. 그리고 break. 기동이 겹치면 먼저 적용된 하나만 적용한다는 규칙이 이 break 한 줄입니다.

구간 밖이면 아무것도 안 하니 자세가 그대로 유지됩니다. 기동이 끝난 뒤 자세를 유지한다는 규칙은 따로 코드가 없고, 안 건드리는 게 곧 유지입니다.

## 04. 한 스텝 함수 f\_StepTarget — ③④⑤ 속도 · 위치 · 위경도

매 스텝 반드시 하는 셋. 자세가 정해졌으니 속도, 속도가 있으니 위치, 위치가 있으니 새 위경도

```
static VOID f_StepTarget(ST_Target *pstTgt, DOUBLE64 dTime, DOUBLE64 dDt)
{
    // ... ㉔ 기동 판단 · 자세 갱신 (앞 장)

    // 3) 속도 변환 (표적 자신의 현재 위경도 기준 NED)
    pstTgt->stVel = f_BodyVelToEcef(pstTgt);

    // 4) 위치 적분 (ECEF)
    pstTgt->stPos.x += pstTgt->stVel.x * dDt;
    pstTgt->stPos.y += pstTgt->stVel.y * dDt;
    pstTgt->stPos.z += pstTgt->stVel.z * dDt;

    // 5) LLA 변환. 출력용이자 다음 스텝의 NED 기준
    pstTgt->stlla = f_Trans_Ecef_To_Lla(pstTgt->stPos.x, pstTgt->stPos.y, pstTgt->stPos.z);
}
```

자동차로 치면 : 시속 720 km 로 0.1 초 가면 20 m. 그걸 600 번 더하면 12 km.  
속도의 방향은 매 스텝 다시 계산하므로 (앞 장의 회전 두 번), 곡선도 이 방식으로 그려진다.



## ③ 속도 변환 — 매 스텝 다시

자세가 안 바뀌어도 부른다.  
표적의 위경도가 바뀌면 NED 축이 바뀌고 ECEF 속도 성분도 바뀐다.

## ④ 위치 적분 — 세 줄

$p \leftarrow p + v \times dt$  를 ECEF x, y, z 에.  
대공 200 m/s 면 한 스텝 20 m, 60 초에 12,000 m.

## ⑤ LLA 변환 — 출력용이자 다음 수평면

f\_Trans\_Ecef\_To\_Lla 로. 이 값이 기록되고,  
다음 스텝 ③ 의 NED 기준이 된다 (7장의 곡률 처리).

## 60 s 뒤 고도 300.019 m

한 스텝 20 m 접선  $\rightarrow 20^2/2R$  씩 뜨고 60 초 동안 쌓여 1.9 cm.  
이론값과 같다

21

## 발표 대본

구현 · 약 70초

뒷부분, 매 스텝 반드시 하는 셋입니다.

셋째, 방금 본 속도 변환 함수를 부릅니다. 자세가 바뀌었든 안 바뀌었든 매 스텝 부릅니다. 표적의 현재 위경도가 매 스텝 달라지니까요.

넷째, ECEF 세 성분에 속도 곱하기 dt 를 더합니다. 위치 적분이 이 세 줄입니다. 대공 표적이면 한 스텝에 20 미터씩, 60초면 12 킬로미터입니다.

다섯째, 좌표변환 함수로 ECEF 를 위경도로 되돌립니다. 이 값이 기록되고, 다음 스텝 셋째 단계의 수평면 기준이 됩니다. 앞에서 본 지구 곡률 문제를 해결하는 게 바로 이 순서입니다. 한 스텝에 20 미터 접선으로 가면 20 제곱 나누기 2R 만큼 뜨는데, 60초 동안 쌓여도 1.9 센티미터라 결과 고도가 300.019 미터입니다.

## 04. 시나리오 전체 — f\_StartScenario · f\_StepScenario

표적 하나의 함수를 플랫폼 + 표적 전부로 뛴 함수 둘. 코어의 시간 축은 이 둘이 전부다

```
// 시퀀스 시작 : 입력 칸으로 상태 칸을 채운다 (한 번)
VOID f_StartScenario(ST_Scenario *pstScn)
{
    INT32 i;

    f_InitTarget(&pstScn->stPlatform);
    for (i = 0; i < pstScn->nTargetCnt; i++)
    {
        f_InitTarget(&pstScn->astTarget[i]);
    }
}

// 시각 dTime 에서 ddt 만큼 전체를 한 스텝 진행 (0.1 초마다 불린다)
VOID f_StepScenario(ST_Scenario *pstScn, DOUBLE64 dTime)
{
    INT32 i;

    f_StepTarget(&pstScn->stPlatform, dTime, pstScn->ddt);
    for (i = 0; i < pstScn->nTargetCnt; i++)
    {
        f_StepTarget(&pstScn->astTarget[i], dTime, pstScn->ddt);
    }
}
```

## 플랫폼 → 표적 순서로 같은 함수

플랫폼은 속력 0 인 카드라 자리만 지킨다.  
표적 수 nTargetCnt 만큼 (최대 10).

## 시각 인자 dTime = k × dt

위치 적분은 시각을 몰라도 되지만  
기동 판단은 지금이 어느 구간인지 알아야 한다.

## 부르는 순서 (UI 든 콘솔이든 같다)

```
1 f_SetAssignment(&scn, bMan)   과제 조건
2 f_StartScenario(&scn)         한 번
3 f_StepScenario(&scn, t)       0.1 초마다
매 스텝 뒤 stUla 를 읽으면 그게 출력이다.
```

플랫폼과 표적이 같은 함수를 지낸다 — 표적이 열 개로 늘어도 함수는 그대로

22

## 발표 대본

구현 · 약 60초

표적 하나를 다루는 함수를 시나리오 전체로 뛴 함수 둘입니다.

f\_StartScenario 는 플랫폼과 표적 전부에 초기화 함수를 부릅니다. 입력 칸으로 상태 칸을 채우는 일이라 한 번만 부릅니다.

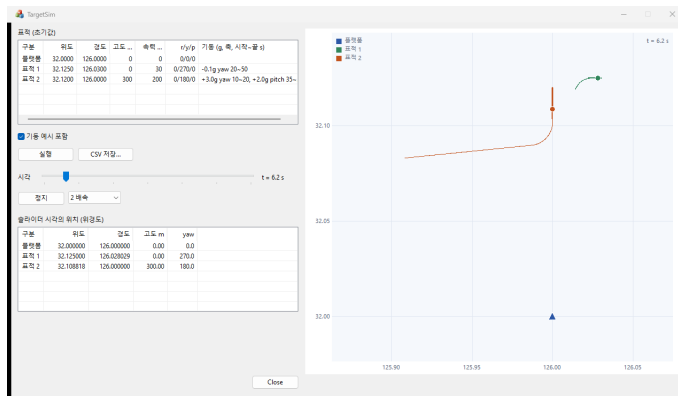
f\_StepScenario 는 시각 하나를 받아 플랫폼과 표적 전부에 한 스텝 함수를 부릅니다. 시각 인자가 필요한 이유는 기동 판단 때문입니다. 위치 적분 자체는 시각을 몰라도 되지만 지금이 어느 기동 구간인지는 알아야 하니까요. 간격은 시나리오 구조체의 dt 를 씁니다.

플랫폼을 표적과 같은 함수로 돌리는 건 앞에서 말씀드린 대로입니다. 속력 0 이라 자리만 지키고, 나중에 배가 움직이는 조건이 오면 카드의 속력과 요만 바꾸면 됩니다.

부르는 순서는 셋입니다. 과제 조건을 채우고, 시작 함수를 한 번, 스텝 함수를 0.1초마다. 코어 쪽은 이게 전부입니다.

## 04. 결과 모으기와 화면 — TargetSimUI (MFC)

MFC 는 계산이 없다. 코어가 낸 위경도를 모아 지도 · 표 · CSV 로 보여 준다



재생 중인 화면 (t = 6.2 s). 점에 붙은 짧은 선이 진행 방향.

## ① 시나리오를 돌리며 위경도를 모은다

[실행] : f\_StartScenario 한 번, f\_StepScenario 를 0.1 초마다.  
매 스텝의 위도 · 경도 · 고도 · yaw 를 표적별로 기록한다.

## ② 지도

기록한 위경도를 그린다. 위도 1° 가 경도 1° 보다 1/cos(위도) 배 길어 축 비율을 맞춘다.  
시각 슬라이더 · 재생으로 임의 시각의 위치를 본다.

## ③ 표와 CSV

슬라이더 시각의 위경도 표.  
[CSV 저장] 은 t, id, 위도, 경도, 고도, yaw 를 한 줄씩 — 과제가 요구한 LLA 출력.

지도의 축 비율을 안 맞추면 선이 원이 타원으로 보인다. 뒤의 파이썬 그림은 이 CSV 로 그렸다.

23

## 발표 대본

구현 · 약 60초

화면입니다. MFC 는 계산이 없습니다.

실행 버튼을 누르면 시작 함수를 한 번 부르고, 스텝 함수를 0.1초마다 부르면서 매 스텝 플랫폼과 표적의 위도, 경도, 고도, 요를 표적별로 기록합니다. 60초가 끝나면 그 기록을 지도에 넘깁니다.

지도는 기록한 위경도를 그립니다. 위도 1도가 경도 1도보다 코사인 위도 분의 1 배 길어서 축 비율을 맞춥니다. 이걸 안 맞추면 선이 원이 타원으로 보입니다. 시각 슬라이더와 재생 버튼으로 임의 시각의 위치를 볼 수 있습니다. 왼쪽 화면이 재생 중인 모습이고, 점에 붙은 짧은 선이 진행 방향입니다.

아래 표가 슬라이더 시각의 위경도이고, CSV 저장은 시각, 번호, 위도, 경도, 고도, 요를 한 줄씩 씁니다. 과제가 요구한 위경도 출력이 이 표와 이 파일이고, 뒤에 나올 파이썬 그림도 이 파일로 그렸습니다.

## 05

시뮬레이션  
결과

1 실행 화면 — 과제 조건

2 궤적과 고도

3 기동 시나리오

4 검증

화면의 값 = CSV 의 값 = 파이썬 그림의 값. 같은 코드가 만든 같은 숫자다.

24

## 발표 대본

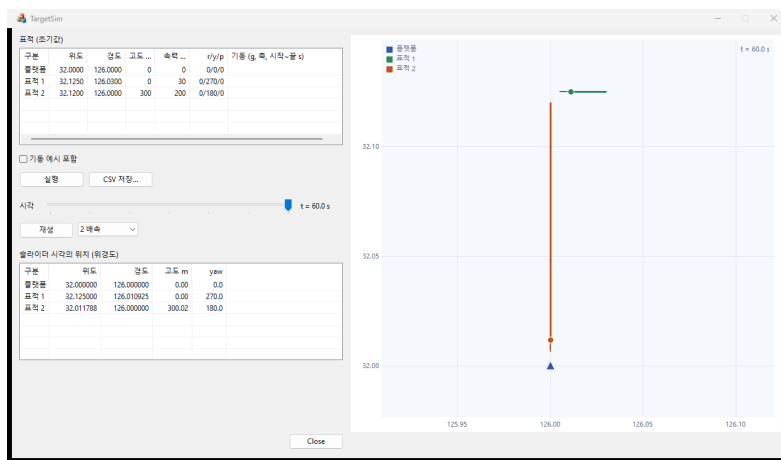
장절 · 약 20초

다섯째 장, 결과입니다. 과제 조건으로 실행한 화면, CSV 로 그린 궤적과 고도, 기동 예시를 컨 시나리오, 그리고 결과가 맞는지 확인한 검증표 순서입니다.

화면의 값, CSV 의 값, 파이썬 그림의 값이 전부 같은 코드가 만든 같은 숫자입니다.

05. 실행 화면 — 과제 조건,  $t = 60\text{ s}$ 

왼쪽 위 표적 카드의 입력 칸, 오른쪽 궤적 지도, 왼쪽 아래 슬라이더 시각의 상태 칸



## 대공 표적 (주황)

60 s 뒤 32.011788°N, 126.0°E, 고도 300.02 m  
플랫폼에서 1,341 m 앞, 고각 1.2° → 12.9°  
정남으로 내려와 삼각형 바로 위에서 끝난다

## 대함 표적 (초록)

60 s 뒤 32.125°N, 126.010925°E  
서쪽으로 1.8 km. 선 길이 차이 = 속력 차이

## 플랫폼 (파랑 삼각형)

32.0°N 126.0°E 에서 자리를 지킨다.  
같은 스텝 함수를 지나지만 속력이 0.

상태 칸의 값이 그대로 [CSV 저장] 으로 나간다 — 과제 요구 "시뮬레이션 결과를 LLA 로 출력"

25

## 발표 대본

결과 · 약 70초

과제 조건 그대로 실행한 화면입니다. 왼쪽 위가 표적 카드의 입력 칸, 오른쪽이 궤적 지도, 왼쪽 아래가 슬라이더 시각의 상태 칸입니다. 슬라이더를 끝까지 보낸 60초 상태입니다.

주황이 대공 표적입니다. 60초 뒤 북위 32.011788도, 동경 126.0도, 고도 300.02 미터. 정남으로 12 킬로미터 내려와 플랫폼 1,341 미터 앞에서 끝납니다. 플랫폼에서 본 고각은 1.2도에서 12.9도까지 올라갑니다. 정면으로 접근하는 조건입니다.

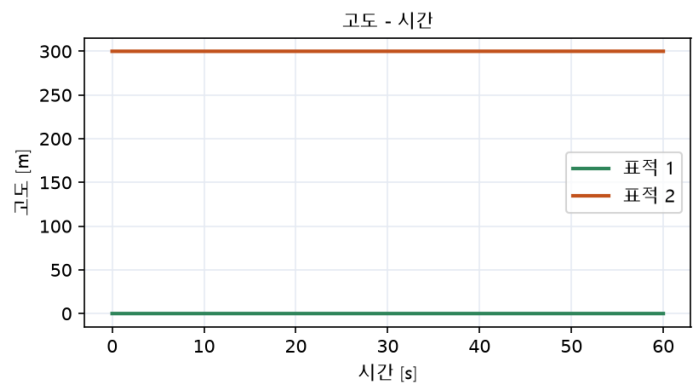
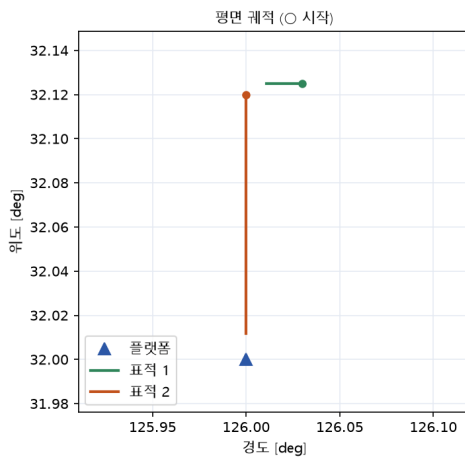
초록이 대함 표적입니다. 60초 뒤 북위 32.125도, 동경 126.010925도. 서쪽으로 1.8 킬로미터 갔습니다. 선 길이 차이가 그대로 속력 차이입니다.

파랑 삼각형이 플랫폼입니다. 같은 스텝 함수를 지나지만 속력이 0 이라 자리를 지킵니다.

상태 칸의 값이 그대로 CSV 저장으로 나갑니다. 과제가 요구한 시뮬레이션 결과를 LLA 로 출력이 이 칸과 이 파일입니다.

## 05. 궤적과 고도 — CSV 를 파이썬으로

평면 궤적은 축 비율을 맞춰서, 고도는 평평해야 한다 (지구 곡률 처리의 증거)



60 s 뒤 고도 300.019 m — 2 cm 의 정체

한 스텝에 20 m 접선으로 가서 20° / 2R 씩 뜨고, 60 초 동안 쌓여 1.9 cm. 이론값과 같다.

매 스텝 수평면을 다시 잡지 않았다면 이 선이 300 → 311 m 로 기울었을 것

26

## 발표 대본

결과 · 약 60초

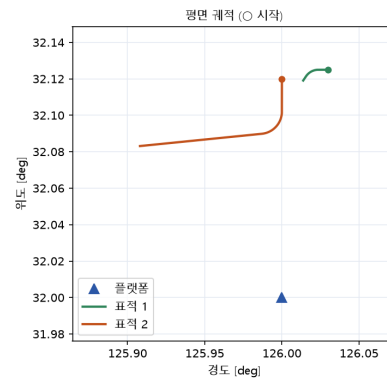
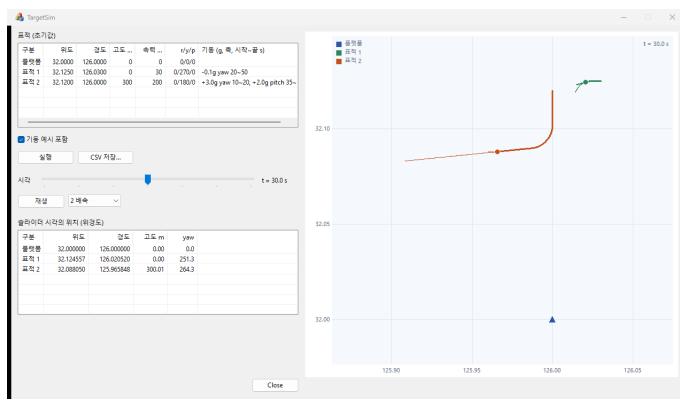
저장한 CSV 를 파이썬으로 그린 그림입니다.

왼쪽이 평면 궤적입니다. 화면의 지도와 같은 그림이고, 축 비율을 맞춰 그렸습니다. 대공 표적이 정남으로 길게, 대함 표적이 서쪽으로 짧게, 두 출발점이 아주 가깝다는 것까지 보입니다.

오른쪽이 고도 시간 그래프입니다. 평평해야 하는 그래프입니다. 대공 표적이 300 미터에서 60초 뒤 300.019 미터. 2 센티미터 오른 건 한 스텝에 20 미터를 접선으로 가서 20 제곱 나누기 2R 씩 뜨는 게 60초 동안 쌓인 값이고, 이론값과 같습니다. 매 스텝 수평면을 다시 잡지 않았다면 이 선이 300 에서 311 미터로 기울었을 겁니다. 지구 곡률 처리가 됐다는 증거가 이 평평한 선입니다.

## 05. 기동 시나리오 — 3 g 우선회, 기수 들기, 좌선회

체크박스를 켜고 실행. 왼쪽은  $t = 30$  s 화면, 오른쪽은 CSV 로 그린 궤적



## 대공 3 g 우선회 (10~20 s)

10초에 84.3° 돌아 서쪽으로.  
궤적 세 점 외접원 1,359.5 m vs  $V^2/(n-g)$  1,359.6 m

## 대공 2 g 기수 들기 (35~40 s)

5초에 28.1° 올라간 뒤 그 각으로 20초.  
60 s 에 고도 2,428.7 m

## 대함 -0.1 g 좌선회 (20~50 s)

30초에 -56.2°, 남서쪽으로.  
반경 917.8 m vs 이론 917.7 m

27

## 발표 대본

결과 · 약 70초

기동 예시를 켜고 실행한 결과입니다. 왼쪽이 30초 시점 화면, 오른쪽이 CSV 로 그린 전체 궤적입니다.

대공 표적은 10초부터 20초까지 3 g 로 우선회합니다. 10초 동안 84.3도 돌아 서쪽을 향합니다. 궤적에서 선회 시작, 중간, 끝 세 점으로 외접원을 구하면 반지름이 1,359.5 미터, V 제곱 나누기 n g 로 계산한 이론 값 1,359.6 미터와 0.01 퍼센트 차이입니다. 35초부터 40초까지는 2 g 로 기수를 듭니다. 5초에 28.1도 올라가고 그 각을 유지해서 60초에 고도 2,428.7 미터입니다.

대함 표적은 20초부터 50초까지 마이너스 0.1 g 로 좌선회합니다. 30초 동안 마이너스 56.2도, 남서쪽을 향합니다. 반지름 917.8 미터, 이론값 917.7 미터입니다.

기동이 끝난 뒤 자세가 유지된다는 것도 궤적에서 보입니다. 선회가 끝난 지점부터 곧게 갑니다.

## 05. 검증 — 다섯 가지로 확인

좌표변환은 틀려도 그럴듯한 값이 나온다 (과제 2의 교훈). 저장한 CSV 로 계산했다

| 항목       | 어떻게 확인하나                                 | 기대값                                | 결과                          |
|----------|------------------------------------------|------------------------------------|-----------------------------|
| ① 이동 거리  | ECEF 위치 차이의 합 vs 속력 × 시간                 | $200 \times 60 = 12,000 \text{ m}$ | 12,000.000 m (차 1e-9 m)     |
| ② 속력 보존  | 매 스텝 $ v_{\text{ecef}} $ vs 입력 속력        | 200 m/s                            | 최소-최대 200.000000 (편차 1e-14) |
| ③ 고도 유지  | 수평 비행 60 s 뒤 고도                          | 300 m                              | 300.019 m (이론 +1.9 cm)      |
| ④ 선회 반경  | 기동 시작-중간-끝 세 점의 외접원 vs $V^2/(n \cdot g)$ | 3 g : 1,359.6 m                    | 1,359.5 m (-0.01 %)         |
| ⑤ 위경도 왕복 | LLA → ECEF → LLA → ECEF 거리               | 1 mm 미만                            | 3e-5 m                      |

## 왜 미리 정하나

코드가 내놓을 값을 먼저 알고 있어야 코드가 맞는지 알 수 있다.  
 ①②③⑤는 순계산한 값과 같고, ④는 기동 코드가 붙은 뒤 됐다.

## 한 번 틀리면 바로 드러나는 실수들

경도·위도 인자 순서 바꿈 → 북위 54° 동경 212°. 속도에 플랫폼 위치 더함 → 6,372 km/s.  
 수평면을 한 번만 잡음 → 고도 311 m.

다섯 칸이 채워졌으니 궤적이 "그림" 이 아니라 "결과" 다

28

## 발표 대본

결과 · 약 70초

지난 과제의 교훈이 하나 있습니다. 좌표변환은 틀려도 그럴듯한 값이 나온다. 이번엔 점이 600개라 눈으로 는 더 못 잡습니다. 그래서 확인 항목을 다섯 개 정해 저장한 CSV 로 계산했습니다.

첫째, 이동 거리. 속력 곱하기 시간이니 대공 표적은 12,000 미터를 가야 합니다. 경로 길이가 12,000.000 미터입니다. 대함은 1,800.000 미터.

둘째, 속력 보존. 매 스텝 ECEF 속도의 크기가 입력 속력과 같아야 합니다. 최소 최대 모두 200.000000, 회전만 했다는 증거입니다.

셋째, 고도 유지. 300.000 에서 300.019 미터, 앞에서 설명한 2 센티미터입니다.

넷째, 선회 반경. 1,359.5 대 1,359.6 미터.

다섯째, 위경도 왕복. 위경도를 ECEF 로 갔다가 되돌리면 같은 값이어야 하는데 오차가 0.03 밀리미터입니다.

오른쪽 아래가 한 번 틀리면 바로 드러나는 실수들입니다. 인자 순서를 바꾸면 북위 54도, 속도에 위치를 더하면 초속 6,372 킬로미터, 수평면을 한 번만 잡으면 고도 311 미터. 다섯 칸이 채워졌으니 이 궤적은 그림이 아니라 결과입니다.

## 정리

- 1 좌표변환 코드 : 함수 넷 — 경도 먼저 · Roll, Yaw, Pitch 순서 · 속도엔 플랫폼 위치 0 · 각도는 라디안
- 2 구조체 : 기동 · 표적 · 시나리오 셋. 표적은 입력 칸 다섯 + 상태 칸 넷, 과제 항목이 칸에 하나씩
- 3 플랫폼도 속력 0 인 표적 카드 — 초기화 · 스텝 함수가 하나로 통일된다
- 4 구현 : 기동 판단 → 자세 갱신 → 속도 변환 (회전 두 번) → 위치 적분 (ECEF) → LLA 변환, 0.1 초마다
- 5 LLA 변환은 출력이자 다음 스텝의 수평면 — 이 순서가 지구 곡률을 처리한다 (고도 300.019 m)
- 6 결과 : 대공 12 km 정면 접근 · 고각  $1.2^\circ \rightarrow 12.9^\circ$ , 대함 1.8 km 서진, 3 g 선회 반경 1,359.5 m
- 7 검증 다섯 : 거리 · 속력 · 고도 · 선회 반경 · 왕복 — 전부 이론값과 일치

질문 환영합니다

코드 · 문서 : [radar-sw-study / Part3\\_TargetSim](#)

29

### 발표 대본

정리 · 약 50초

정리하겠습니다. 과제가 제출하라고 한 네 가지에 맞춰 말씀드리면,

좌표변환 코드는 함수 넷을 썼습니다. 경도 먼저, 롤 요 피치 순서, 속도엔 플랫폼 위치 0, 각도는 라디안. 이 네 규약을 컴파일해서 확인했습니다.

표적 구조체는 기동, 표적, 시나리오 셋입니다. 표적은 입력 칸 다섯과 상태 칸 넷으로 나뉘고, 과제 항목이 칸에 하나씩 대응됩니다. 플랫폼도 카드 한 장입니다.

구현은 한 스텝 다섯 단계입니다. 기동 판단, 자세 갱신, 속도 변환은 회전 두 번, 위치 적분은 ECEF 에서 한 줄, 위경도 변환은 출력이자 다음 수평면. 이걸 0.1초마다 반복합니다.

결과는 검증 다섯 가지로 확인했습니다. 거리, 속력, 고도, 선회 반경, 왕복. 전부 이론값과 맞습니다.

이상입니다. 질문 환영합니다.

발표에서는 말하지 않는다. 질문이 들어올 때 보는 메모.

Q1. 오일러 적분이면 오차가 쌓이지 않나?

한 스텝 안에서 속도가 상수라 직진 구간은 정확하다. 선회 구간에서는 자세를 먼저 돌리고 그 방향으로 0.1 초를 가므로 다각형으로 원을 근사하는 셈인데, 3 g 에서 한 스텝  $0.84^\circ$  라 반지름 오차가 0.01 % 다. 필요하면 dt 를 줄이거나 중점법으로 바꾸면 되고, 구조는 그대로다.

Q2. 왜 NED 를 플랫폼이 아니라 표적 자리에 잡나?

자세각은 표적 자신의 수평면 기준으로 정의된 값이다. 플랫폼 자리의 NED 로 돌리면 13 km 떨어진 곳의 수평면이라 아주 작게 기운다. 이번 거리에서는  $0.1^\circ$  차이지만 원칙이 다르다.

Q3. 고도가 타원체고인가 해발고도인가?

참고 코드가 WGS-84 타원체 기준이라 타원체고다. 이 근처에서 해발고도와 약 20 m 차이가 나지만 과제는 좌표변환 일관성만 보므로 그대로 두었다.